

National Teachers College

629 J. Nepomuceno St., Quiapo, Manila

School of Arts, Sciences and Technology



Hands-On Activity: The Anti-Hacker Lab

(Week 11 Security: CSRF and XSS Protection)

PC 21 | Advanced Web Development

Custorio, April Nicole

3.2 BSIT

Prof. Edward James V. Grageda

May 2026

Hands-On Activity: The Anti-Hacker Lab

TASK 01: Enable CSRF Protection

In Task 1, CSRF protection was enabled by modifying the Filters.php configuration file and adding the CSRF filter: 'csrf',

app/Config/Filters.php

```

Filters.php x SecurityLab.php security_form.php App.php
app > Config > Filters.php > Filters
16 class Filters extends BaseFilters
    4 references
73 public array $globals = [
74     'before' => [
75         'csrf',
76     ],
77     'after' => [
78     ],
79 ];
  
```

After enabling this, the application automatically required a valid CSRF token for all POST requests.

Code Implementation (SecurityLab.php):

```

Filters.php SecurityLab.php x security_form.php App.php
app > Controllers > SecurityLab.php > ...
1 <?php
2
3 namespace App\Controllers;
4
5 use CodeIgniter\Controller;
6
7 class SecurityLab extends Controller
8 {
9     0 references | 0 overrides
10    public function index()
11    {
12        return view('security_form');
13    }
14
15    0 references | 0 overrides
16    public function submit()
17    {
18        $name = $this->request->getPost('name');
19
20        return view('security_form', [
21            'name' => $name
22        ]);
23    }
24 }
  
```

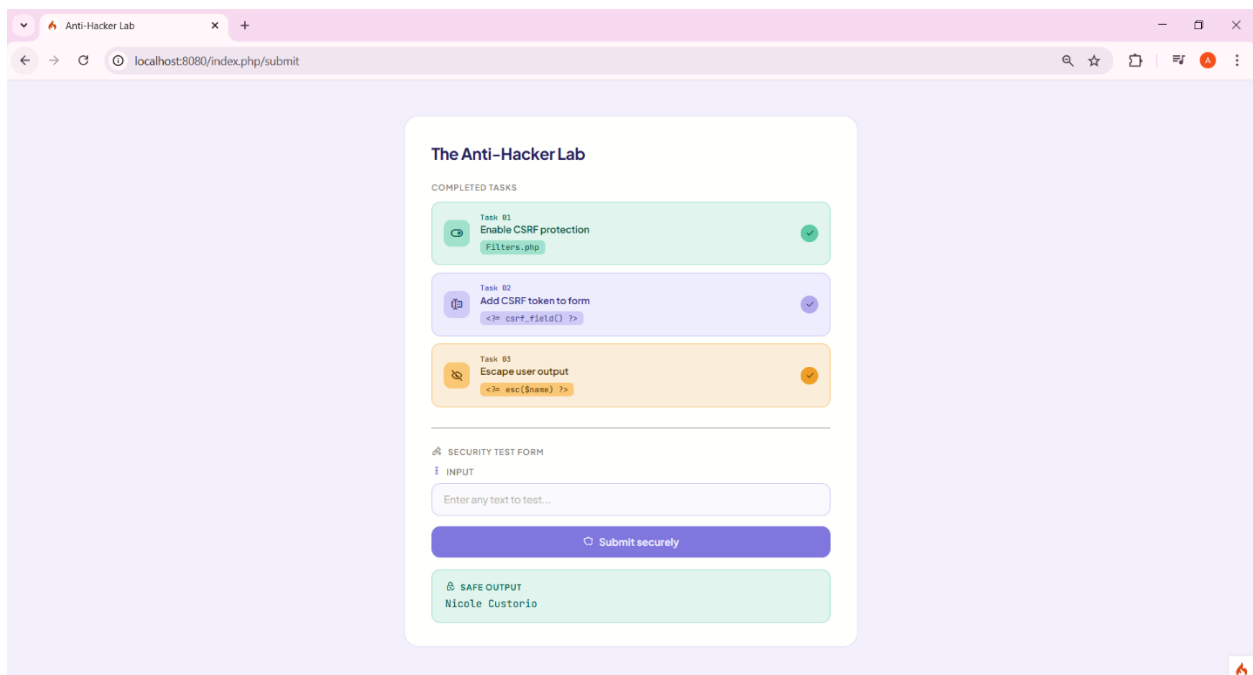


Week 11: Security: CSRF and XSS Protection

```
Filters.php SecurityLab.php security_form.php X App.php
app > Views > security_form.php
2 <html>
256 <body>
258 <div class="card">
326
327 <?php if (isset($name)): ?>
328 <div class="output-box">
329 <div class="output-label">
330 <i class="ti ti-lock-check"></i>
331 Safe output
332 </div>
333 <p class="output-value"><?= esc($name) ?></p>
334 </div>
335 <?php endif; ?>
```

```
Filters.php SecurityLab.php security_form.php Routes.php X
app > Config > Routes.php
1 <?php
2
3 namespace Config;
4
5 use CodeIgniter\Config\Services;
6
7 $routes = Services::routes();
8
9 $routes->get('/', 'SecurityLab::index');
10 $routes->post('/submit', 'SecurityLab::submit');
```

Output:



Week 11: Security: CSRF and XSS Protection

TASK 02: Add CSRF Token to Form

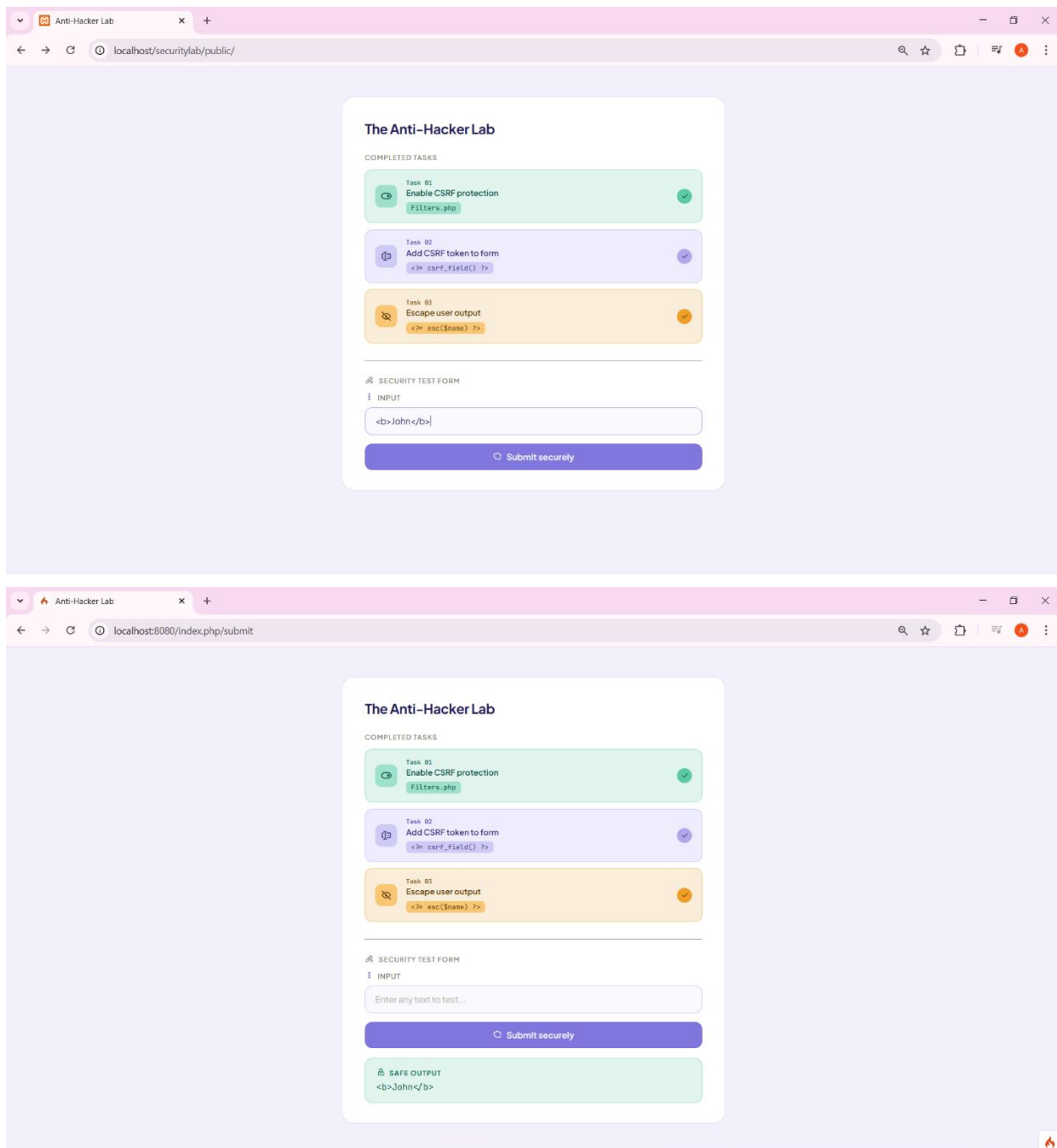
For Task 2, a CSRF token was added inside the HTML form using the following code: `<?= csrf_field() ?>`

This generated a hidden input field containing a security token that is validated on form submission.

During Testing:

With `<?= csrf_field() ?>` Enabled:

XSS Test Input: `<b>John</b>` displays the literal string `<b>John</b>`, not bold text.



The Anti-Hacker Lab

COMPLETED TASKS

- Task 01: Enable CSRF protection (Filters.php) ✓
- Task 02: Add CSRF token to form (`<?= csrf_field() ?>`) ✓
- Task 03: Escape user output (`<?= esc($name) ?>`) ✓

SECURITY TEST FORM

INPUT

`<b>John</b>`

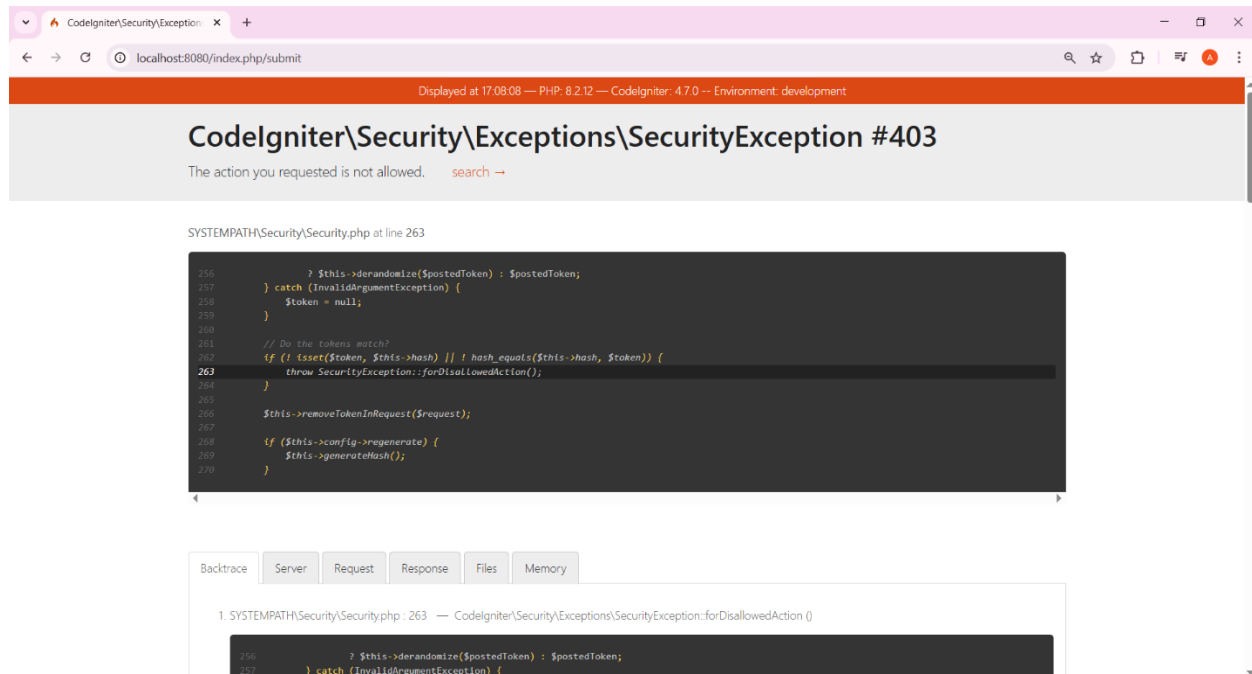
Submit securely

SAFE OUTPUT

`<b>John</b>`

Week 11: Security: CSRF and XSS Protection

Without `<?= csrf_field() ?>`: Removed `<?= csrf_field() ?>` from the form temporarily.



During CSRF testing, the form submission fails with 403 if `csrf_field()` is missing from the form.

Week 11: Security: CSRF and XSS Protection

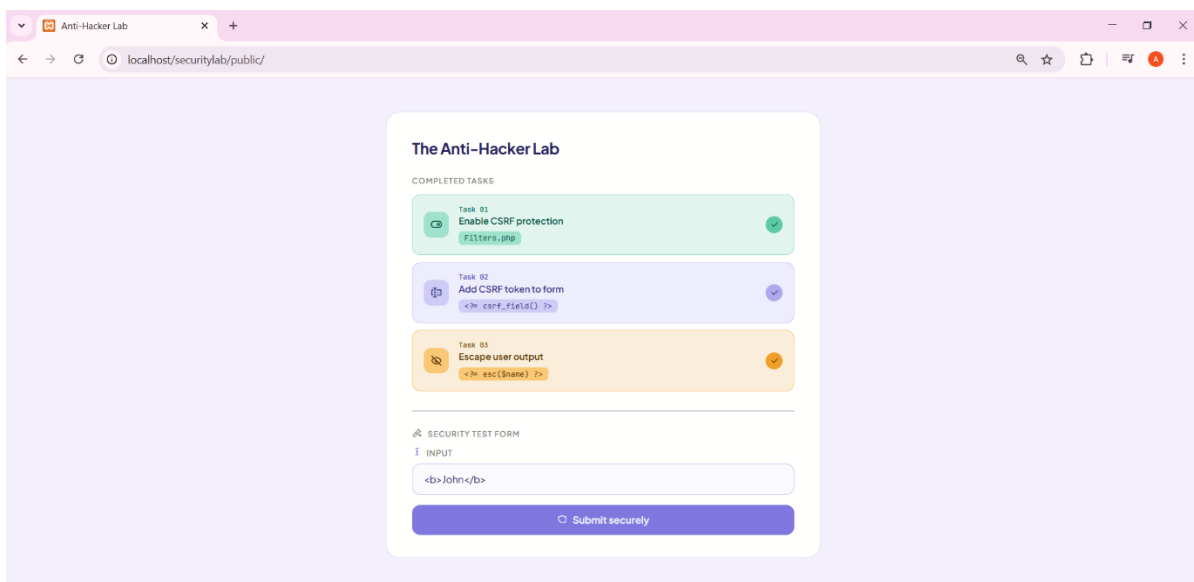
TASK 03: XSS Protection (Escaping User Input)

For Task 3, XSS protection was implemented by sanitizing user input using CodeIgniter's `esc()` function:

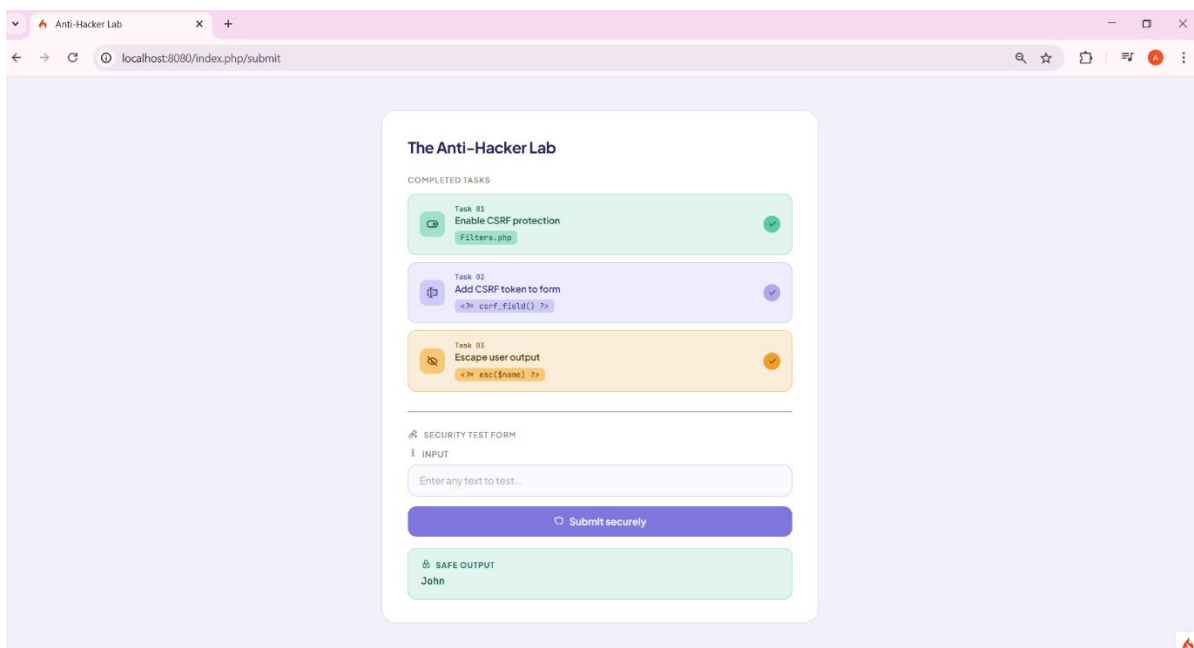
```
<?= esc($user_input) ?>
```

During Testing 1:

With `esc()` Enabled: The `<b>` tags were converted to HTML entities, so `<b>John</b>` appeared as plain text on the screen.



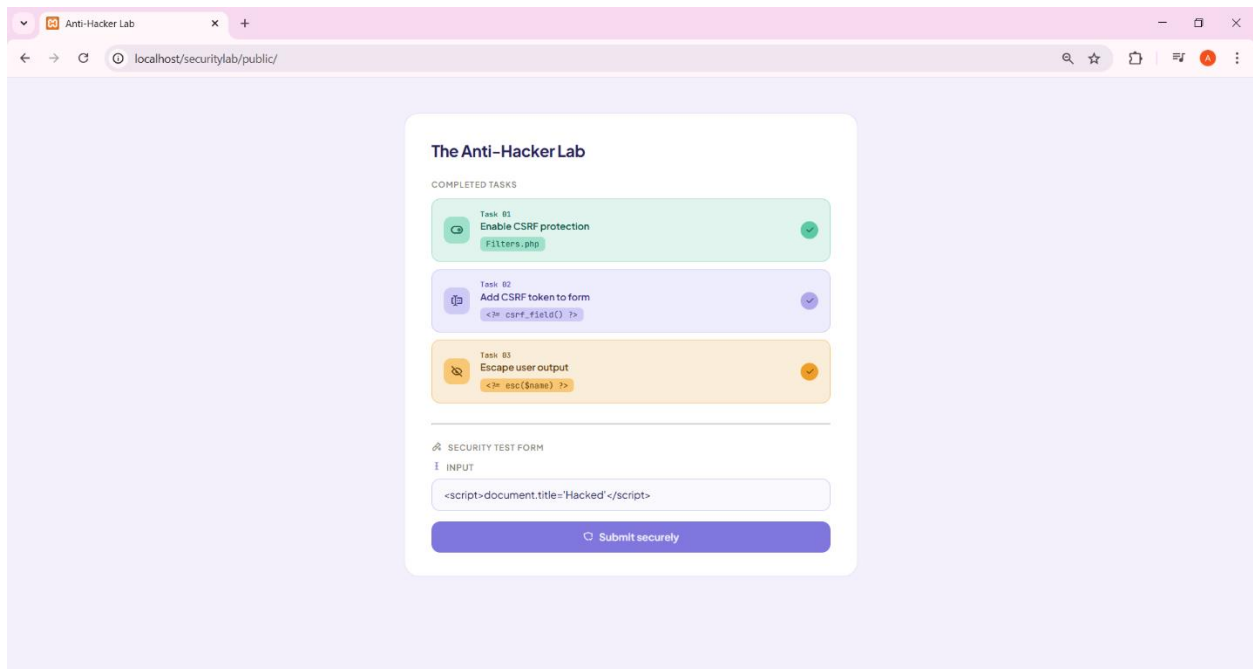
Without `esc()`: The `<b>` tags were not escaped, so the browser interpreted them as HTML, making "John" appear in bold text.



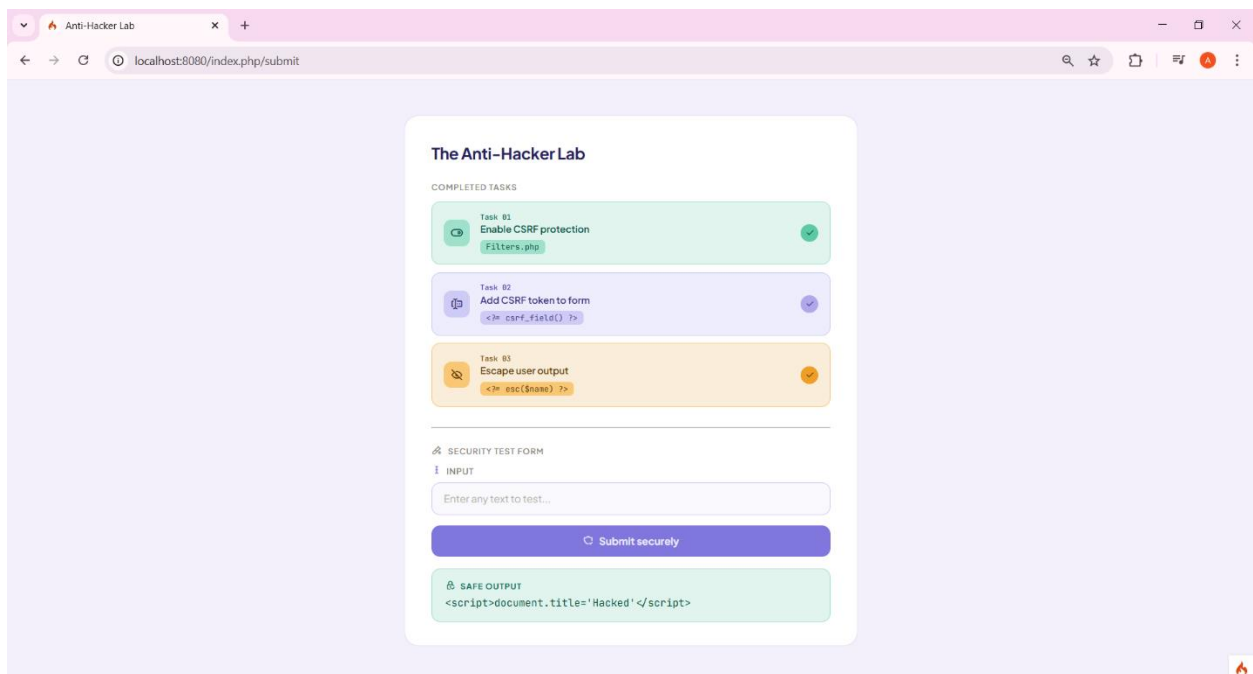
Week 11: Security: CSRF and XSS Protection

During Testing 2:

Input: `<script>document.title='Hacked'</script>`

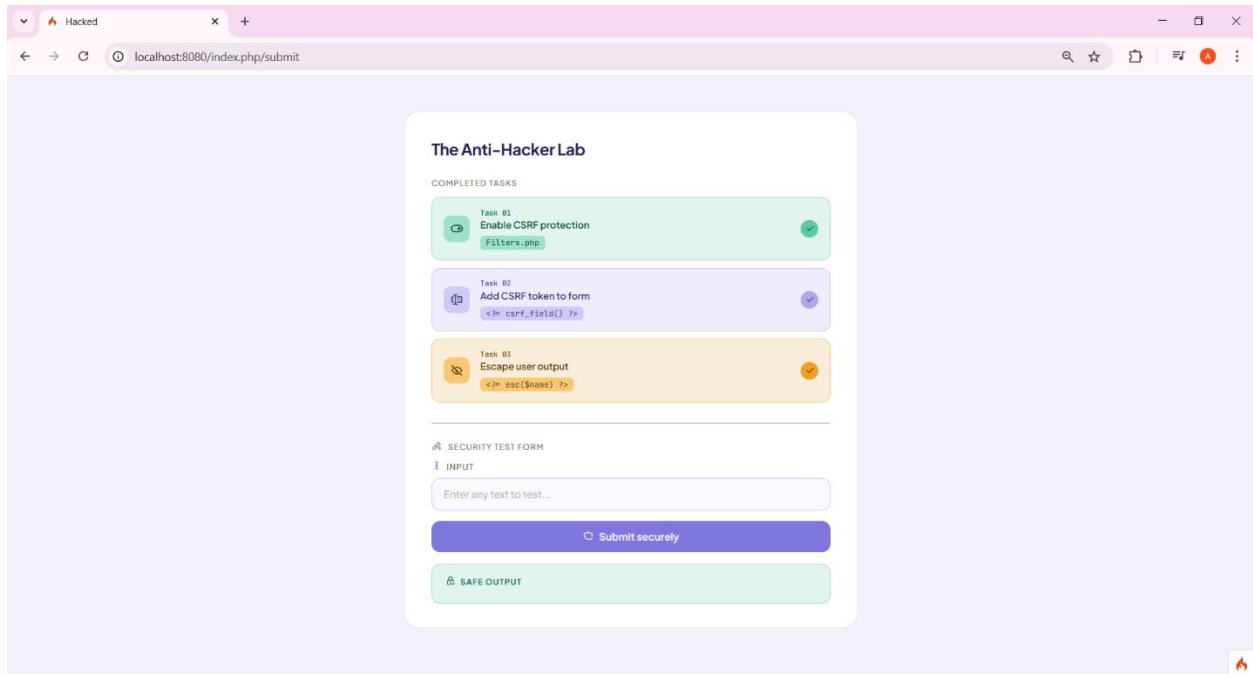


Submitted with `esc()` enabled: Tab title stays "Anti-Hacker Lab - Result":

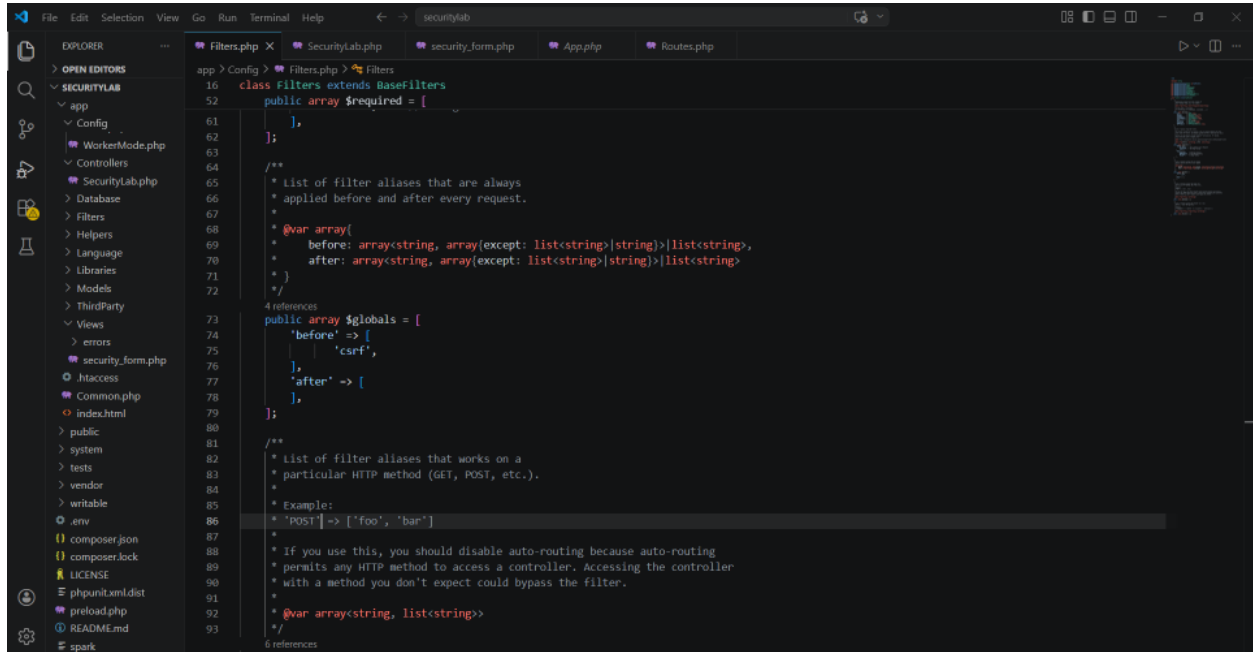


Week 11: Security: CSRF and XSS Protection

Temporarily removed `esc()` from `security_form.php`: The script executed, tab title changed to "Hacked".



Folder Structure:



Discussion Answers**Q1: Can CSRF attacks happen with GET requests?**

Yes, CSRF attacks can happen with GET requests, especially if the application performs actions like deleting data, updating profiles, or changing settings using GET instead of POST. Since GET requests can be triggered simply by visiting a link or loading an image/script, an attacker can trick a user into unknowingly sending a request to a trusted website while they are logged in. This is why sensitive actions should never use GET requests and should always use POST with CSRF protection.

Q2: What if an attacker guesses the CSRF token?

If an attacker successfully guesses a valid CSRF token, the protection can be bypassed. However, in real applications, this is extremely difficult because CSRF tokens are designed to be long, random, and unique per session and sometimes per request. They are generated using secure random algorithms, making them practically impossible to predict. Additionally, many frameworks invalidate tokens after use or rotate them frequently, further reducing the chance of exploitation.

Q3: Why isn't validating input enough to stop XSS?

Validating input alone is not enough to prevent XSS because validation only checks if the input follows expected rules such as format or length, but it does not guarantee that the data is safe when displayed in the browser. An attacker can still inject malicious scripts that pass validation rules. XSS prevention requires output encoding or escaping like using `esc()` in CodeIgniter so that any HTML or JavaScript is treated as plain text instead of executable code. This ensures that even if harmful input is stored in the system, it will not execute in the user's browser.